SA 1: Quality Assurance Report

Dela Cruz, Cloie Sofia R. | *Zabala, Ralf Cedric T.*

I. Dataset Overview

This quality assurance report documents the controls applied to ensure that the constructed transportation dataset is consistent, usable for modeling, and traceable for reproducibility. The dataset is derived from a trimmed roadside video and converted into image frames for object detection, with annotations provided as bounding boxes across 7 classes. In its current release, the dataset contains 1005 images with 0 unannotated frames, ensuring that all exported samples contribute meaningful supervision during training. The class composition reflects the project's goal of representing locally frequent Philippine road users, and the dataset is prepared for YOLO-style detection through a standardized export format that pairs each image with a corresponding label file (Redmon et al., 2016).

Item	Value
Raw video source	Roadside self-recorded video (1 clip) captured along Kalayaan Avenue, Makati, Metro Manila, around 3 PM; originally 45 minutes, 60 FPS, 1080p, trimmed to 30 minutes for dataset extraction and annotation.
Frame extraction	Roboflow video-to-frames extraction with sampling at 2 frames/second, producing 3,599 extracted frames for inspection and labeling.
Annotation target	Philippine road-user and vehicle instance detection in an urban intersection/road segment, emphasizing locally frequent classes commonly absent or merged in non-Philippine datasets.
Classes	Bicycle; Car; Jeepney; Pedestrian; SUV; Tricycle; Truck (7 classes).
Annotation type	Object detection labels using bounding boxes aligned with YOLO-style training requirements.
Labeling scope control	Frames with heavy obstructions were removed; distant vehicles that could not be confidently distinguished were not

	labeled. Labeling was prioritized within the visually reliable region (up to the pedestrian lanes and intersection areas).
Final version used	Roboflow dataset version compiled with 1,005 source images, 7 classes, 0 unannotated; split into 705 train / 154 validation / 146 test; preprocessing: Auto-Orient = Applied, Resize = Stretch to 640×640, Filter Null = Require all images to contain annotations; Augmentation = Off.

II. Data Framing Size and Preprocessing Controls

To support consistent training inputs, the dataset version was compiled using fixed preprocessing controls in Roboflow. Auto-orient was enabled to correct any unintended rotation introduced during video capture or frame extraction, ensuring that spatial orientation remains consistent across images. All frames were resized using stretching to 640×640, aligning the dataset with common YOLO input configurations and enforcing a uniform resolution across training, validation, and testing subsets (Redmon & Farhadi, 2018). This standardization reduces variance in model input shape and ensures that performance comparisons are based on a stable and repeatable preprocessing pipeline. At the same time, the group recognizes that stretching may alter aspect ratios; therefore, this choice is explicitly documented as part of the dataset version so that future versions may compare alternative resizing strategies (e.g., letterboxing) under controlled conditions.

To prevent annotation integrity issues, Filter Null was applied such that only images containing annotations were included in the released version. This preprocessing constraint guarantees that the dataset does not contain unlabeled frames that could function as false negatives during training. The dataset split configuration was also preserved within the version to support unbiased evaluation, with 705 images for training, 154 for validation, and 146 for testing. These controls ensure that the same samples remain in the same subsets across experiments unless a new dataset version is created.

III. Dataset Preprocessing

Purpose

The augmentation stage of the preprocessing pipeline was designed as an offline procedure to address class imbalance within the YOLO-format object detection dataset (Johnson & Khoshgoftaar, 2019). Rather than applying transformations dynamically during model training or modifying the validation and test partitions, the pipeline generates additional training samples for underrepresented classes through label-preserving image transformations applied in conjunction with coordinated bounding box updates. This approach increases minority-class exposure in the training distribution while preserving evaluation integrity, as the validation and test splits remain entirely unchanged throughout the augmentation process (Shorten & Khoshgoftaar, 2019).

Implementation

The augmentation pipeline operates by loading the existing dataset from a baseline "no augmentation" directory and writing the resulting balanced version to a designated output folder. The procedure begins by computing per-class bounding box annotation counts across the training split to identify which classes fall below the chosen representation target. For each underrepresented class, source training images containing at least one instance of that class are iteratively selected, and augmented variants are produced until the additional annotation requirement is satisfied.

```
aug_counter = 0

for cls_id, cls_name in enumerate(CLASS_NAMES):
    needed = TARGET_COUNT - current_counts.get(cls_id, 0)
    if needed <= 0:
        print(f"[{cls_name}] Already at or above target. Skipping.")
        continue

    source_images = class_to_images[cls_id]
    if not source_images:
        print(f"[{cls_name}] No source images found! Skipping.")
        continue

    print(
        f"[{cls_name}] Generating ~{needed} additional annotations "
        f"from {len(source_images)} source images..."
    )

    added = 0
    attempts = 0
    max_attempts = needed * 5  # safety limit

    while added < needed and attempts < max_attempts:
        img_path, lbl_path = random.choice(source_images)
        result = augment_image(img_path, lbl_path, augmentation)
        attempts += 1

        if result is None:
            continue

        aug_img, aug_bboxes, aug_cls_ids = result

        # Count how many of the TARGET class are in this augmented sample
        cls_in_aug = sum(1 for c in aug_cls_ids if c == cls_id)
        if cls_in_aug == 0:
            continue
```

```
        # Save
        aug_name = f"aug_{cls_id}_{aug_counter:05d}"
        aug_counter += 1

        aug_img_bgr = cv2.cvtColor(aug_img, cv2.COLOR_RGB2BGR)
        cv2.imwrite(str(aug_img_dir / f"{aug_name}.jpg"), aug_img_bgr)
        write_yolo_labels(
            str(aug_lbl_dir / f"{aug_name}.txt"), aug_bboxes, aug_cls_ids
        )

        added += cls_in_aug

    print(f"  → Added {added} annotations ({aug_counter} total aug images so far)")
```

Both the transformed image and its corresponding updated YOLO label file are written to the output directory, yielding a reproducible, disk-backed balanced training set. This offline balancing strategy was preferred over on-the-fly augmentation at

training time, as it produces a fixed, inspectable dataset state that can be versioned and audited independently of the training configuration.

```python
# --- Configuration ---
DATASET_ROOT = Path("NoAug")
BALANCED_ROOT = Path("Balanced")

SPLITS = ["train", "valid", "test"]

# Load data.yaml for class names
with open(DATASET_ROOT / "data.yaml", "r") as f:
    data_yaml = yaml.safe_load(f)

CLASS_NAMES = data_yaml["names"]
NUM_CLASSES = len(CLASS_NAMES)

print(f"Dataset root : {DATASET_ROOT}")
print(f"Number of classes: {NUM_CLASSES}")
print(f"Class names : {CLASS_NAMES}")
```

```
Dataset root : NoAug
Number of classes: 7
Class names : ['Bicycle', 'Car', 'Jeepney', 'Pedestrian', 'SUV', 'Tricycle', 'Truck']
```

Augmentation Pipeline Configuration

Augmentation was implemented using Albumentations Compose with bounding box parameters configured for YOLO format, ensuring that all geometric transformations are applied consistently to both image pixels and their associated annotations (Buslaev et al., 2020). A minimum visibility threshold was enforced throughout, so bounding boxes rendered degenerate or severely cropped by a given transformation are automatically discarded rather than retained as erroneous labels.

```python
# --- 5.1 Define augmentation pipeline ---
augmentation = A.Compose(
    [
        A.HorizontalFlip(p=0.5),
        A.VerticalFlip(p=0.1),
        A.RandomBrightnessContrast(
            brightness_limit=0.25, contrast_limit=0.25, p=0.6
        ),
        A.HueSaturationValue(
            hue_shift_limit=15, sat_shift_limit=30, val_shift_limit=20, p=0.5
        ),
        A.GaussNoise(var_limit=(10, 50), p=0.3),
        A.GaussianBlur(blur_limit=(3, 5), p=0.2),
        A.RandomScale(scale_limit=0.15, p=0.3),
        A.ShiftScaleRotate(
            shift_limit=0.1,
            scale_limit=0.15,
            rotate_limit=10,
            border_mode=cv2.BORDER_CONSTANT,
            p=0.5,
        ),
    ],
    bbox_params=A.BboxParams(
        format="yolo", label_fields=["class_ids"], min_visibility=0.3
    ),
)
```

The configured transforms and their parameters are as follows:

- HorizontalFlip (p = 0.5)
- VerticalFlip (p = 0.1)
- RandomBrightnessContrast: brightness_limit = 0.25, contrast_limit = 0.25 (p = 0.6)
- HueSaturationValue: hue_shift_limit = 15, sat_shift_limit = 30, val_shift_limit = 20 (p = 0.5)
- GaussNoise: var_limit = (10, 50) (p = 0.3)
- GaussianBlur: blur_limit = (3, 5) (p = 0.2)
- RandomScale: scale_limit = 0.15 (p = 0.3)
- ShiftScaleRotate: shift_limit = 0.1, scale_limit = 0.15, rotate_limit = 10, border_mode = BORDER_CONSTANT (p = 0.5)
- Bounding box handling: format = "yolo", min_visibility = 0.3, label_fields = ["class_ids"]

Quality Assurance Controls

Several quality assurance controls were embedded within the augmentation process to reduce the risk of label corruption and preserve the validity of the resulting dataset.

First, bounding box geometry is preserved through label-consistent transformation. Because annotations are passed through Albumentations with explicit YOLO-format bounding box parameters, geometric operations such as flipping, scaling, and rotation are applied simultaneously to pixel data and their corresponding labels, preventing silent annotation drift.

Second, the minimum visibility rule of 0.3 functions as a guardrail against degenerate annotation cases arising from extreme transformations. Any bounding box whose visible area falls below 30% of its original extent after a transformation is discarded, thereby reducing label noise that would otherwise be introduced by severely cropped or partially occluded augmented instances.

Third, the augmentation pipeline is restricted exclusively to the training partition. The validation and test splits are left entirely unchanged, which is a critical control for preserving an unbiased estimate of detector generalization during evaluation.

Fourth, upon completion of the augmentation phase, the pipeline recomputes and visualizes per-class annotation counts before and after balancing. This post-generation verification step serves as a quantitative sanity check to confirm that the augmentation procedure achieved the intended redistribution of class representation.

Inter-Annotator Agreement and Professional Oversight

INTER-ANNOTATOR AGREEMENT AND LABEL VALIDATION LETTER

February 28, 2026

To Whom It May Concern,

We are Computer Science students of **Mapúa University** under the course **CSS182-3/BM2 - Data Science 3**, hereby certify that we conducted a structured annotation and validation process for our video-derived dataset in compliance with the course requirements.

Our selected domain is **Transportation**, specifically **vehicle classification**, and the dataset annotations were created for the following seven (7) labels:

- bicycle
- car
- suv
- jeepney
- truck
- tricycle
- pedestrian

As part of our **quality assurance protocol**, we implemented an **inter-annotator review process** to verify the correctness and consistency of labels applied to extracted video frames. The annotation outputs were reviewed by at least two (2) annotators and checked for agreement in terms of:

- **Class label correctness** (proper object category assigned)
- **Consistency of label usage** across similar objects and scenes
- **Handling of ambiguous cases** (such as occlusion, motion blur, low-light frames, partial object visibility)
- **Annotation quality standards** based on our agreed labeling guidelines

After review and reconciliation of disagreements, the group confirms that the final labels included in the submitted structured annotated dataset have been **checked, validated, and approved as correct to the best of our knowledge** under the defined annotation guidelines of this project.

This letter serves as documentation that an inter-annotator validation step was completed as part of the dataset preprocessing and quality assurance stage.

We affirm that this process was performed in good faith and that all corrections identified during the review were incorporated into the final dataset version submitted for course requirements.

Respectfully submitted,

Cloie Sofia R. Dela Cruz
3rd Year Mapúa University Student

Ralf Cedric T. Zabala
3rd Year Mapúa University Student

Guilbert B. Lingbaoan
Police Master Sergeant

Reyzen Aldrin A. Ramos
Licensed Professional Teacher

Outputs



The balancing procedure produces an enlarged training set alongside updated dataset configuration files, all written to the "Balanced/" output directory. The pipeline records the number of augmented images generated and reports updated per-class counts upon completion, providing a traceable record of the changes introduced between the input dataset and the balanced output. This documentation supports reproducibility by making the augmentation outcome inspectable independently of the training run.

IV. References

Buslaev, A., Iglovikov, V. I., Khvedchenya, E., Parinov, A., Druzhinin, M., & Kalinin, A. A. (2020). Albumentations: Fast and flexible image augmentations. *Information*, *11*(2), 125. https://doi.org/10.3390/info11020125

Johnson, J. M., & Khoshgoftaar, T. M. (2019). Survey on deep learning with class imbalance. *Journal of Big Data*, *6*(1), 1–54. https://doi.org/10.1186/s40537-019-0192-5

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 779–788. https://arxiv.org/abs/1506.02640

Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*. https://arxiv.org/abs/1804.02767

Shorten, C., & Khoshgoftaar, T. M. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data*, *6*(1), 1–48. https://doi.org/10.1186/s40537-019-0197-0